ML Assignment 2 - Final Submission

Generated 2026-08-15

ML Assignment 2 - Final Submission

Student: [PASTE_YOUR_NAME_AND_BITS_ID_HERE] **Course:** Machine Learning (AIMLCZG565)
Submission Deadline: 18-Aug-2026, 23:59 PM

This document follows the mandatory submission order specified in the assignment: (1) GitHub Repository Link, (2) Live Streamlit App Link, (3) BITS Virtual Lab execution screenshot, (4) full README.md content.

---

1. GitHub Repository Link

[PASTE_GITHUB_REPO_LINK_HERE]

The repository contains: - Complete source code (`app.py` , `model/train_models.py`) - `requirements.txt` - A clear `README.md` - Test data used in the experiments (`test_data.csv`)

---

2. Live Streamlit App Link

[PASTE_LIVE_STREAMLIT_APP_LINK_HERE]

Deployed via Streamlit Community Cloud from the `main` branch, app entry point `app.py` . Confirmed to open an interactive frontend when clicked.

---

3. Screenshot - Execution on BITS Virtual Lab

[INSERT_BITS_LAB_SCREENSHOT_HERE — e.g. screenshot]

(One screenshot showing the assignment — e.g. `python model/train_models.py` or the Streamlit app running — executed on the BITS Virtual Lab environment.)

---

4. README.md Content

> The full content of the repository's `README.md` is reproduced below, as required by Section 2, item 4 of the assignment.

Credit Risk Classification - ML Assignment 2

a. Problem Statement

Banks need to decide whether a loan applicant is a **good credit risk** (likely to repay) or a **bad credit risk** (likely to default) before sanctioning a loan. This project builds and compares six supervised classification models that predict credit risk from an applicant's financial and demographic attributes, and exposes them through an interactive Streamlit web app so a user can upload test data, pick a model, and inspect its performance.

b. Dataset Description

- **Name:** Statlog (German Credit Data) - numeric version
- **Source:** UCI Machine Learning Repository - https://archive.ics.uci.edu/dataset/144/statlog+german+credit+data (file used: `german.data-numeric`)
- **Instances:** 1000 loan applicants (>= 500 required)
- **Features:** 24 numeric attributes (>= 12 required). The original dataset has 20 attributes (7 numeric + 13 categorical); Strathclyde University's numeric release indicator-codes the categorical attributes, expanding them into 24 purely numeric columns (`Attribute_1` ... `Attribute_24`) so that algorithms requiring numeric input can use it directly. The underlying source attributes cover: checking-account status, loan duration, credit history, purpose of loan, credit amount, savings balance, employment duration, installment rate, personal status, other debtors/guarantors, residence duration, property owned, age, other installment plans, housing type, number of existing credits, job type, number of dependents, telephone ownership, and foreign-worker status.
- **Target:** `CreditRisk` - recoded from the original labels (1 = Good, 2 = Bad) to **1 = Good credit risk (positive class)**, **0 = Bad credit risk**. Class distribution is imbalanced (~70% Good / ~30% Bad), which is typical of real credit-risk data and is reflected in the observations below.
- **Note:** UCI's documentation for this dataset also specifies an asymmetric cost matrix - misclassifying a *Bad* applicant as *Good* is penalized 5x more heavily than the reverse - which is why accuracy alone is not a sufficient metric here and MCC/Recall matter.

c. GitHub Repository Link

[PASTE_GITHUB_REPO_LINK_HERE]

d. Models Used

All 6 models below were trained on the **same** 80/20 stratified train-test split (`random_state=42`) of the German Credit dataset, with features standardized using `StandardScaler` (fit on the training split

only). Metrics were computed on the held-out 20% test split (200 rows) - the same data exported as `test_data.csv` for use in the Streamlit app.

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.7250	0.7492	0.7972	0.8143	0.8057	0.3360
Decision Tree	0.6800	0.7413	0.7836	0.7500	0.7664	0.2599
kNN	0.7100	0.7023	0.7470	0.8857	0.8105	0.2266
Naive Bayes	0.6600	0.6748	0.7903	0.7000	0.7424	0.2518
Random Forest (Ensemble)	0.7900	0.8055	0.8025	0.9286	0.8609	0.4617
SVM (RBF kernel)	0.7400	0.7613	0.7821	0.8714	0.8243	0.3371

(Positive class = 1 = Good credit risk. Numbers regenerated by running `model/train_models.py`; see that script for full methodology.)

Observations on model performance

ML Model Name	Observation about model performance
Logistic Regression	Solid linear baseline (Accuracy 0.725, MCC 0.336). Works reasonably well because several attributes (checking-account status, credit history, savings) are ordinal and roughly monotonic with risk, but it cannot capture interaction effects (e.g., loan purpose combined with employment duration), which caps its ceiling below the ensemble model.
Decision Tree	Lowest AUC/MCC among the non-Naive-Bayes models (Accuracy 0.680, MCC 0.260) even with depth capped at 6 to limit overfitting. A single tree partitions on one feature at a time and is high-variance on a modestly-sized (1000-row), noisy dataset like this one - small changes in the training split would likely change its splits noticeably.
kNN	Highest Recall (0.8857) of the non-ensemble models, i.e. it is very good at correctly flagging "Good" applicants, but its Precision (0.7470) and MCC (0.2266, the lowest overall) are the weakest - it is comparatively more prone to False Positives (calling a Bad applicant Good), which is exactly the costly error per UCI's stated cost matrix. Performance is sensitive to feature scaling (which we apply) and to the curse of dimensionality across 24 features.
Naive Bayes	Weakest overall (Accuracy 0.660, AUC 0.675) because Gaussian NB assumes all 24 features are conditionally independent given the class - an assumption that clearly does not hold here, since several of the 24 columns are indicator-encodings derived from the same original categorical attribute (e.g., multiple columns tracing back to "purpose of loan") and are therefore correlated by construction. Still useful as a fast, interpretable probabilistic baseline.
Random Forest (Ensemble)	**Best model on every single metric** (Accuracy 0.790, AUC 0.8055, Precision 0.8025, Recall 0.9286, F1 0.8609, MCC 0.4617). Averaging 200 decorrelated trees reduces the variance problem that hurts the single Decision Tree, while still capturing non-linear feature interactions that Logistic Regression and Naive Bayes miss - without needing a conditional-independence or linearity assumption. It also achieves the best Recall alongside strong Precision, i.e. it best balances catching Good applicants without over-approving Bad ones.
SVM (RBF kernel)	Second-best model overall (Accuracy 0.740, AUC 0.7613, F1 0.8243, MCC 0.3371) - clearly ahead of the linear/naive baselines and close behind Random Forest. The RBF kernel implicitly maps the 24 scaled features into a higher-dimensional space, letting it capture non-linear boundaries similar to the Random Forest, but as a single dense-margin classifier it is more sensitive to the class imbalance and does not benefit from the variance-reduction that comes from averaging many trees.
Overall Winner for your dataset?	**Random Forest (Ensemble)** - it dominates all 5 other models on all 6 evaluation metrics on this dataset, with SVM as the strongest runner-up, making Random Forest the clear choice for deployment.

How to Reproduce

```
# 1) create environment
pip install -r requirements.txt

# 2) train all 6 models, generate test_data.csv + metrics_comparison.csv
python model/train_models.py

# 3) run the Streamlit app locally
streamlit run app.py
```

Repository Structure

```
ML_Assignment2_CreditRisk/
├── app.py                          # Streamlit app
├── requirements.txt
├── README.md
├── test_data.csv                   # held-out test split (200 rows) for the app
├── data/
│   └── german.data-numeric         # raw UCI dataset
└── model/
    ├── train_models.py             # trains all 6 models + saves metrics/models
    ├── metrics_comparison.csv      # generated metrics table
    └── saved_models/               # pickled models + scaler used by app.py
```